Process Management & Scripting

ICCS121 Syskills | MUIC

Lecture 03 | Processes, signals, job control, and bash scripts

Who Am I?

- **ผศ.ดร. ทรงพล ตีระกนก** (Asst. Prof. Dr. Songpon Teerakanok)
- Assistant Professor, MUIC — Computer Science
- Founder, CYBERSKILLS Professional Training Center
- Research: Cybersecurity, Network Security, AI Security
- Email: **songpon.tee@mahidol.ac.th**

⚠️ Disclaimer

- Slides are a **summary** — not a complete reference
- Always verify commands in your own terminal
- Some behavior may differ by OS version or distro
- For authoritative info: `man` pages, official docs

Agenda

1. Program vs. Process (Recap)
2. Process Management Commands
3. Creating and Running Programs
4. Shell Scripting Basics
5. Branching — Conditional Statements
6. Loops
7. Practical Scripting Example

Program vs. Process (Recap)

The foundation of process management

Key Distinctions

Program

- Binary data stored **on disk**
- **Passive** entity — a file
- Example: `/bin/bash`

Process

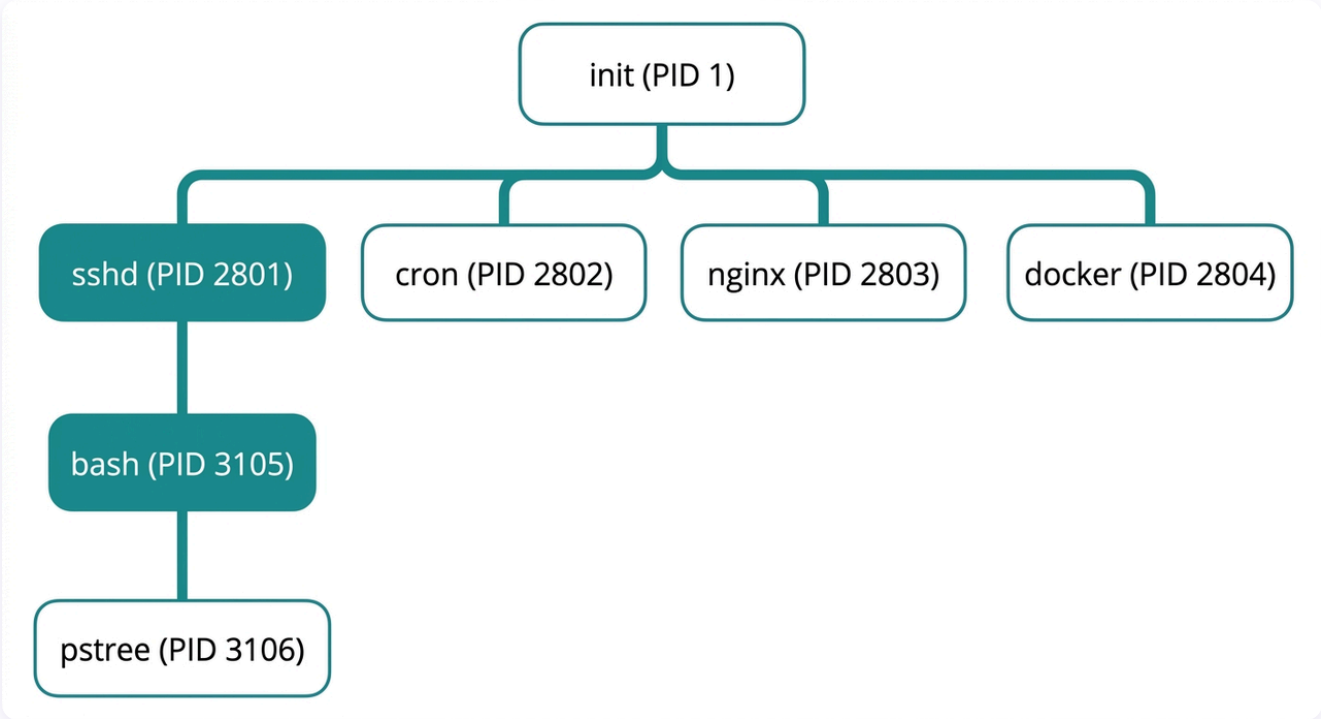
- Running instance **in memory**
- **Active** — has PID, state, memory
- Has: PID, PPID, memory, open files
- States: running, sleeping, stopped, zombie

```
# Two terminals, each running bash = two processes, one program
$ ps aux | grep bash
student   1234  ...  /bin/bash    # Process 1
student   5678  ...  /bin/bash    # Process 2
```

The Process Tree

```
$ pstree
init─┬─acpid
     ├─cron
     ├─sshd───sshd───bash───pstree     # you are here
     ├─nginx──4*[nginx]
     └─docker──5*[{docker}]
```

- Every process has a **parent** (except PID 1 = init/systemd)
- Your shell creates **child processes** for every command
- `pstree -p` shows PIDs alongside names



src: ai-generated

Process Management Commands

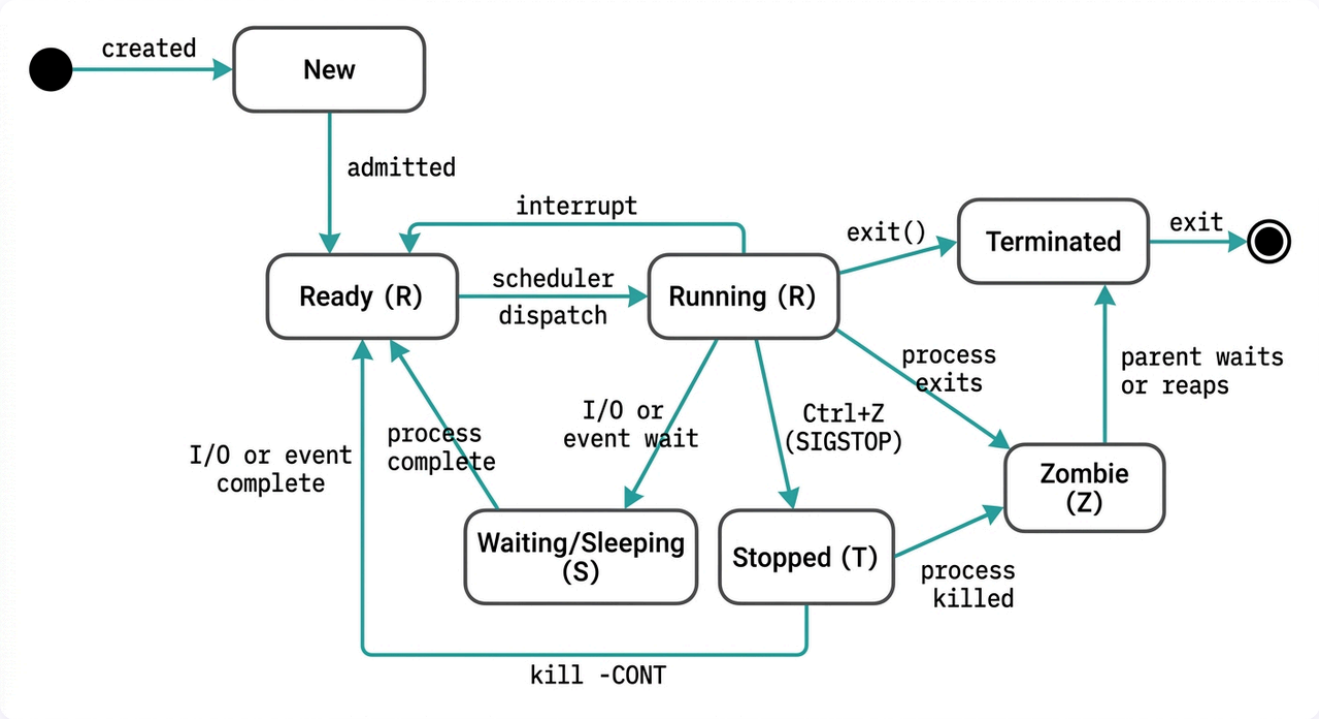
Monitor, control, and terminate processes

Viewing Processes — `ps`

```
ps                  # processes in current terminal only
ps aux              # ALL processes on the system
ps -ef              # ALL processes (POSIX syntax)
```

Column	Meaning	Example
USER	Process owner	`student`
PID	Process ID	`1234`
%CPU	CPU usage	`2.3`
%MEM	Memory usage	`0.5`
STAT	Process state	`S` (sleeping)
COMMAND	Command that started it	`/usr/bin/python3 app.py`

Process States



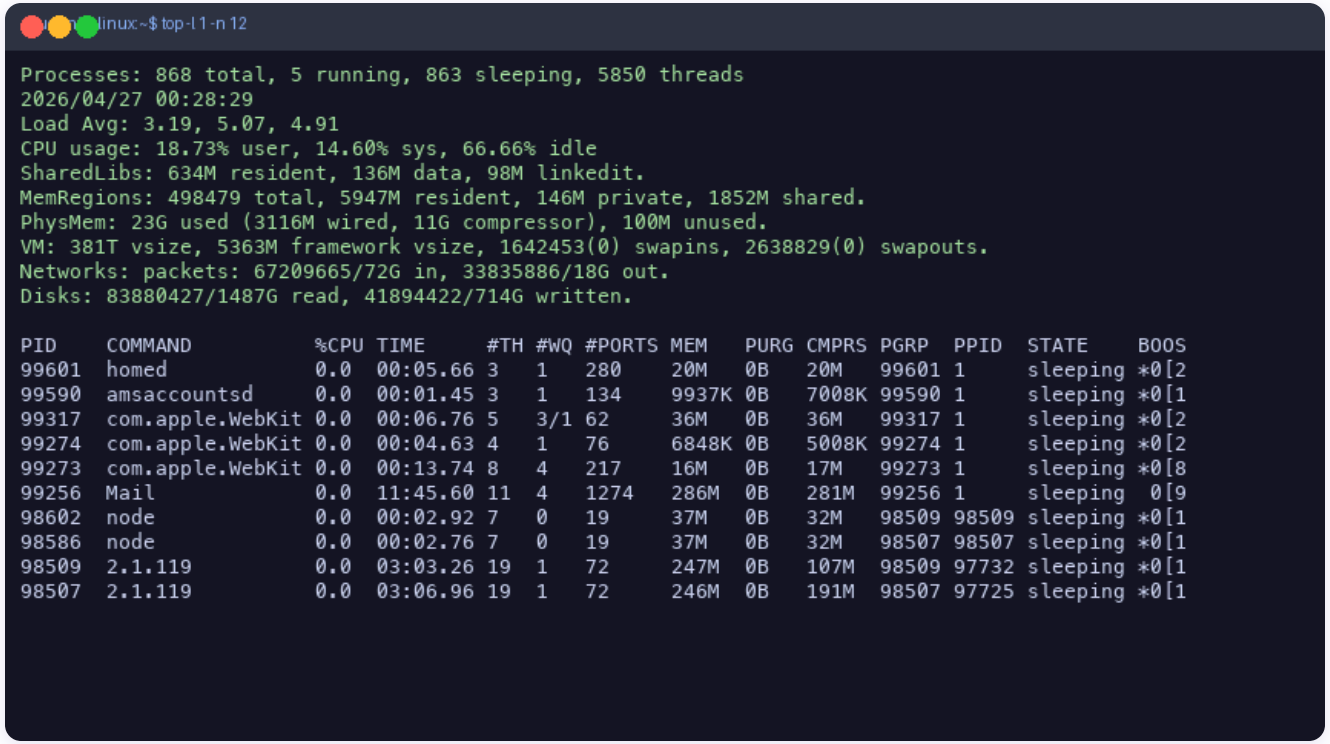
src: ai-generated

State	Symbol	Meaning
Running	`R`	Running or in the run queue
Sleeping	`S`	Waiting for an event (I/O, timer)
Uninterruptible Sleep	`D`	Waiting for I/O (cannot be interrupted)
Stopped	`T`	Paused by Ctrl+Z or a signal
Zombie	`Z`	Terminated but parent has not collected exit status

Real-Time Monitoring — `top`

- Updates every few seconds with live CPU/memory stats
- Key shortcuts inside `top`:

Key	Action
`q`	Quit
`k`	Kill a process (enter PID)
`M`	Sort by memory usage
`P`	Sort by CPU usage



src: terminal screenshot

Killing Processes

```
kill 1234          # SIGTERM (graceful termination)
kill -9 1234       # SIGKILL (forced — cannot be caught!)
kill -l            # list all signal names
```

Signal	Number	Can Be Caught?	Use Case
SIGTERM	15	Yes	Default — polite "please exit"
SIGKILL	9	**No**	Last resort — instant death
SIGINT	2	Yes	Ctrl+C — interrupt
SIGSTOP	19	**No**	Pause (like Ctrl+Z)
SIGCONT	18	Yes	Resume a stopped process
SIGHUP	1	Yes	Terminal closed / hangup

Job Control

Command/Key	Action
`command &`	Start in the background
`Ctrl+Z`	Pause foreground process
`jobs`	List all jobs in current shell
`fg` or `fg %2`	Bring job to foreground
`bg`	Resume stopped job in background

Job Control — Workflow Example

```
$ sleep 100 &            # start in background
[1] 5678                 # job [1], PID 5678

$ jobs                   # list jobs
[1]+  Running  sleep 100 &

$ fg %1                  # bring to foreground
sleep 100
^Z                       # Ctrl+Z: pause it
[1]+  Stopped  sleep 100

$ bg %1                  # resume in background
[1]+ sleep 100 &

$ kill %1                # kill job #1
```

Creating and Running Programs

From source code to execution

How the Shell Finds Programs (PATH)

```
echo $PATH
# /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin

which ls          # /bin/ls — found in /bin/
```

- Shell searches each PATH directory left to right
- Current directory (`.`) is **not** in PATH — security measure
- Not in PATH? Use explicit path:

```
./myscript.sh              # relative path
/home/student/myscript.sh  # absolute path
```

Binary vs. Script Executables

Type	Content	How It Runs
Binary	Compiled machine code (ELF)	CPU executes directly
Script	Plain text + interpreter directive	Interpreter reads and executes

```
file /bin/ls          # ELF 64-bit LSB pie executable, x86-64
file myscript.sh      # Bourne-Again shell script, ASCII text
```

The Shebang Line

```
#!/bin/bash              # interpreted by bash
#!/usr/bin/python3       # interpreted by Python 3
#!/usr/bin/env python3   # portable: finds python3 via PATH
```

- **Must be the first line** of the script
- `#!` tells the OS which interpreter to use
- Without it, the current shell tries to interpret (may fail)

Making Scripts Executable

```
vim backup.sh          # 1. create the script
chmod +x backup.sh     # 2. add execute permission
./backup.sh            # 3. run it
```

- New files are **not executable** by default
- Without `chmod +x` → `Permission denied`
- Alternative: `bash backup.sh` (no chmod needed)

Shell Scripting Basics

Automating tasks with bash

Your First Script

```
#!/bin/bash
# backup.sh — a simple backup script

SRC="/home/student/documents"
DEST="/home/student/backup"
DATE=$(date +%Y-%m-%d)

mkdir -p "$DEST"
cp -r "$SRC" "$DEST/documents-$DATE"
echo "Backup completed: $DEST/documents-$DATE"
```

Variables

```
# Assignment — NO spaces around =
name="Alice"
count=42
greeting="Hello, $name"     # variables expand in double quotes

# Reading — prefix with $
echo $name                   # Alice
echo "$greeting"             # Hello, Alice
echo "Count is ${count}."    # Count is 42.

# Read user input
read -p "Enter your name: " username
echo "Welcome, $username"
```

Special Variables

Variable	Meaning	Example Value
`$#`	Number of arguments	`3`
`$0`	Script name	`./backup.sh`
`$1` .. `$9`	Positional arguments	`$1` = first arg
`$@`	All arguments (separate strings)	`"arg1" "arg2"`
`$?`	Exit status of last command	`0` = success
`$$`	PID of current script	`12345`

`"$@"` vs `$@` — Why Quoting Matters

```
#!/bin/bash
# demo.sh — called as: ./demo.sh "first arg" second

# Without quotes — spaces split arguments
for arg in $@; do echo "[$arg]"; done
# [first] [arg] [second]

# With quotes — preserves original quoting
for arg in "$@"; do echo "[$arg]"; done
# [first arg] [second]
```

- Always use `"$@"` to preserve argument boundaries

Quoting Rules

Type	Syntax	Variable Expansion	Use Case
Unquoted	`Hello $name`	Yes	Simple cases (risky)
Double quotes	`"Hello $name"`	Yes	Strings with variables/spaces
Single quotes	`'Hello $name'`	**No** (literal)	Literal strings

```
name="World"
echo Hello $name        # Hello World
echo "Hello $name"      # Hello World
echo 'Hello $name'      # Hello $name (literal)
```

Branching — Conditional Statements

Making decisions in scripts

Comparison Operators (1 of 2)

Numeric Comparisons

Operator	Meaning
`-eq`	Equal
`-ne`	Not equal
`-lt`	Less than
`-gt`	Greater than
`-le`	Less or equal
`-ge`	Greater or equal

Comparison Operators (2 of 2)

String Comparisons

Operator	Meaning
`=` or `==`	Equal
!=	Not equal
`<`	Less than (alphabetical)
`>`	Greater than (alphabetical)
`-z`	String is empty
`-n`	String is not empty

if / elif / else

```bash
#!/bin/bash
num_a=400
num_b=200

if [ $num_a -lt $num_b ]; then
    echo "$num_a is less than $num_b!"
elif [ $num_a -eq $num_b ]; then
    echo "$num_a equals $num_b!"
else
    echo "$num_a is greater than $num_b!"
fi
```

- `if ... then ... fi` — always close with `fi`
- `elif` = "else if" — as many as needed
- `;` before `then` — or put `then` on next line

Parameter Validation Pattern

```bash
#!/bin/bash
if [ "$#" -ne 1 ]; then
    echo "Illegal number of parameters"
    echo "Usage: $0 <filename>"
    exit 1
fi

if [ "$1" == "your string" ]; then
    echo "YES"
else
    echo "NO"
fi
```

- Always validate arguments before using them
- `exit 1` terminates the script with error status

File and String Tests (1 of 2)

Test	True When
`-f file`	File exists and is a regular file
`-d dir`	Directory exists
`-e path`	Path exists (any type)
`-r file`	File is readable
`-w file`	File is writable
`-x file`	File is executable
`-s file`	File exists and is not empty
`-z "$str"`	String is empty

File and String Tests (2 of 2)

Test	True When
`-n "$str"`	String is not empty

Loops

Repeating actions

for Loops

```bash
# Iterate over a list
for i in 1 2 3; do
    echo $i
done

# Iterate over files
for file in *.txt; do
    echo "Processing $file"
    wc -l "$file"
done

# C-style for loop
for ((i=0; i<5; i++)); do
    echo "Iteration $i"
done
```

while Loops

```bash
#!/bin/bash
counter=0

while [ $counter -lt 3 ]; do
    let counter+=1
    echo $counter
done
# Output: 1, 2, 3
```

```bash
# Infinite loop with break
while true; do
    read -p "Enter command (quit to exit): " cmd
    if [ "$cmd" = "quit" ]; then
        break
    fi
    echo "You typed: $cmd"
done
```

Reading Files in Loops

```bash
# Read file line by line (preserves spaces)
while IFS= read -r line; do
    echo "Line: $line"
done < input.txt

# Process each word in a file
for word in $(cat items.txt); do
    echo -n "$word" | wc -c
done
```

- `while IFS= read -r` = line-by-line (preserves spaces)
- `for word in $(cat ...)` = word-by-word (splits on spaces)

Practical Scripting Example

Putting it all together

Backup Script — Full Example

```
#!/bin/bash
# backup_home.sh — backup with date stamp

# Validate arguments
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <backup_directory>"
    exit 1
fi

BACKUP_DIR="$1"
DATE=$(date +%Y-%m-%d_%H%M%S)
SRC="$HOME"
DEST="$BACKUP_DIR/home-backup-$DATE"
```

Backup Script — Full Example (2)

```
# Check if backup directory exists
if [ ! -d "$BACKUP_DIR" ]; then
    echo "Error: $BACKUP_DIR does not exist"
    exit 1
fi

# Perform backup
echo "Starting backup of $SRC to $DEST..."
mkdir -p "$DEST"
cp -r "$SRC"/* "$DEST/" 2>/dev/null
```

Backup Script — Full Example (3)

```
# Check result
if [ $? -eq 0 ]; then
    echo "Backup completed: $DEST"
    echo "Files backed up: $(ls "$DEST" | wc -l)"
else
    echo "Backup failed with exit code $?"
    exit 1
fi
```

Key Takeaways

1. **Program** = file on disk; **Process** = running instance with PID
2. Process states: **R**(unning), **S**(leeping), **T**(stopped), **Z**(ombie)
3. Always try `kill PID` (**SIGTERM**) before `kill -9` (**SIGKILL**)
4. Job control: `&`, `Ctrl+Z`, `fg`, `bg`, `jobs`
5. Scripts need `#!/bin/bash` (shebang) + `chmod +x`
6. Variable assignment: **no spaces** around `=`
7. Always use `"$@"` (quoted) to preserve argument boundaries

Thank You

ICCS121 L03: Process Management & Scripting

Q&A

All questions are welcome

EMAIL

songpon.tee@mahidol.ac.th

OFFICE HOURS

By appointment · MUIC